

Policy Gradient

算法详解笔记

千喜 Qx

2026/06/05

Contents

1	背景与动机	1
1.1	从 Value-Based 到 Policy-Based	1
1.2	策略梯度的核心思想	1
2	核心定义与符号体系	1
2.1	轨迹 (Trajectory) 的定义	1
2.2	轨迹概率分解	2
2.3	目标函数	2
3	梯度推导 (逐步)	2
3.1	第一步: 对目标函数求导	3
3.2	第二步: 对数导数技巧 (Likelihood Ratio Trick)	3
3.3	第三步: 展开 $\log p(\tau \theta)$	3
3.4	第四步: 得到原始梯度公式	4
4	因果律与梯度简化	4
4.1	问题: 过去奖励不应影响当前动作	4
4.2	严格证明	4
4.3	简化后的梯度公式	6
5	REINFORCE 算法	6
5.1	从公式到算法	6
5.2	完整算法流程 (伪代码)	7
5.3	直观理解	7
5.4	REINFORCE 的方差问题	7

6	方差减小技术——Baseline	8
6.1	全正/全负奖励也能工作的原因	8
6.2	Baseline 的正确性：不影响期望值	8
6.3	改进后的梯度估计	8
6.4	为什么 baseline 常用 $V^\pi(s)$ 而不是 $Q^\pi(s, a)$?	8
7	连续动作空间的处理	9
7.1	离散 vs 连续的本质区别	9
7.2	神经网络输出分布参数	9
7.3	高斯策略的对数概率梯度	9
7.4	为什么不需要遍历无穷动作空间?	10
8	数值小例子	10
8.1	环境设定	10
8.2	初始策略参数	11
8.3	单条轨迹采样	11
8.4	梯度计算逐项展开	11
8.5	参数更新	12
8.6	更新前后策略对比	12
9	代码实现与实战项目	12
9.1	PyTorch REINFORCE 完整代码	12
9.2	推荐实战项目作业	15
9.2.1	作业 1：车杆平衡升级版——引入连续动作控制（入门）	15
9.2.2	作业 2：经典街机小游戏——像素输入下的 Pong（进阶）	15
9.2.3	作业 3：手写高维连续动作挑战——双足机器人行走（高阶）	15
10	工程实践：标准化与调参	16
10.1	批次级基线减法与方差缩放	16
10.1.1	减去均值：动态基线的物理引入	16
10.1.2	除以标准差：梯度平稳化	16
10.1.3	举例：动手算一遍	17
10.2	PG 算法是 On-policy 吗?	17
10.3	学习率 (Learning Rate) 的设置讲究	18
10.4	折扣因子 (Gamma) 的设置讲究	18
A	附录	18
A.1	核心公式汇总	18
A.2	符号表	19

1 背景与动机

1.1 从 Value-Based 到 Policy-Based

在强化学习中，算法大致分为两类，如表 1 所示：

Table 1: 强化学习算法分类对比

方法	代表算法	核心思想
Value-Based	Q-Learning、DQN	学习状态或状态-动作的价值 $Q(s, a)$ ，再根据价值选择动作
Policy-Based	Policy Gradient、REINFORCE	直接参数化策略 $\pi_{\theta}(a s)$ ，通过梯度上升优化策略

Value-Based 的局限：

- 对于连续动作空间， $Q(s, a)$ 需要对无限个动作求极值，无法直接使用
- 策略本身是确定性的 (ϵ -greedy)，难以表达随机策略
- 当动作空间较大时，softmax 计算代价高

Policy-Based 的优势：

- 自然处理连续动作空间（输出分布参数）
- 直接优化最终目标（策略的期望奖励）
- 收敛更平滑，避免了对价值函数的过估计问题

1.2 策略梯度的核心思想

策略梯度的核心思想是**直接对策略的参数 θ 求梯度**，通过梯度上升使得目标函数 $\mathcal{J}(\theta)$ 增大：

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \mathcal{J}(\theta) \quad (1)$$

其中 $\nabla_{\theta} \mathcal{J}(\theta)$ 指向策略改善的方向。

核心结论

Policy Gradient 的本质就是“好的动作多做，坏的动作少做”。通过轨迹奖励 G_t 来加权调整动作选择的概率分布。

2 核心定义与符号体系

2.1 轨迹 (Trajectory) 的定义

轨迹 (Trajectory) τ 是指一个完整的交互序列：

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T, a_T, r_{T+1}, s_{T+1}) \quad (2)$$

其中 T 是轨迹长度（可以是有限的或无限的）。

注 2.1 (关于奖励下标的说明)— 在时间步 $t = 0$ 时，智能体处于状态 s_0 并刚刚决定采取动作 a_0 。动作还未执行、环境还未给出反馈，因此没有 r_0 。奖励 r_t 严格来说是“在状态 s_{t-1} 下采取动作 a_{t-1} 后获得的奖励”，即 $r_{t+1} \sim P(r|s_t, a_t)$ ，奖励的下标总是比动作晚一步。

2.2 轨迹概率分解

一条轨迹 τ 在策略 π_θ 下的概率为：

$$p(\tau|\theta) = p(s_0) \prod_{t=0}^T \pi_\theta(a_t|s_t) \cdot P(s_{t+1}|s_t, a_t) \quad (3)$$

符号说明：

- $p(s_0)$ ：初始状态分布（由环境决定，与 θ 无关）
- $\pi_\theta(a_t|s_t)$ ：在状态 s_t 下选择动作 a_t 的概率（策略网络输出）
- $P(s_{t+1}|s_t, a_t)$ ：状态转移概率（由环境决定，与 θ 无关）

注意：只有 $\pi_\theta(a_t|s_t)$ 与参数 θ 有关，其余均为环境参数。

2.3 目标函数

我们希望最大化所有可能轨迹的期望奖励：

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \int p(\tau|\theta) R(\tau) d\tau \quad (4)$$

其中 $R(\tau) = \sum_{t=0}^T r_t$ 是整条轨迹的总奖励。

符号说明：

- $\mathbb{E}_{\tau \sim \pi_\theta}[\cdot]$ ：在策略 π_θ 生成的轨迹上取期望
- $R(\tau)$ ：轨迹的总奖励，可以是折扣的或非折扣的

3 梯度推导（逐步）

我们的目标是找到参数 θ 的梯度 $\nabla_\theta \mathcal{J}(\theta)$ ，通过梯度上升来更新参数。

3.1 第一步：对目标函数求导

$$\nabla_{\theta} \mathcal{J}(\theta) = \nabla_{\theta} \int p(\tau|\theta) R(\tau) d\tau = \int \nabla_{\theta} p(\tau|\theta) R(\tau) d\tau \quad (5)$$

这里将梯度 ∇_{θ} 推进到积分内部（可以交换，因为积分是对 τ 的， ∇_{θ} 作用于 θ ）。

思考 3.1— 能不能说模型输出的轨迹 τ 跟参数 θ 有关，那为什么不对 $R(\tau)$ 也求梯度？

答

不能。因为 θ 只是决定一个轨迹出现的概率大小，但是对某条特定轨迹而言，它的奖励是由环境规则决定的，是固定的。

3.2 第二步：对数导数技巧 (Likelihood Ratio Trick)

利用恒等式：

$$\nabla_{\theta} p(\tau|\theta) = p(\tau|\theta) \nabla_{\theta} \log p(\tau|\theta) \quad (6)$$

代入第一步的结果：

$$\nabla_{\theta} \mathcal{J}(\theta) = \int p(\tau|\theta) \nabla_{\theta} \log p(\tau|\theta) \cdot R(\tau) d\tau \quad (7)$$

这正是期望的形式：

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\nabla_{\theta} \log p(\tau|\theta) \cdot R(\tau)] \quad (8)$$

为什么这样做？ 直接对 $p(\tau|\theta)$ 求导很困难，轨迹概率涉及复杂的策略概率乘积。采用对数导数技巧后， $p(\tau|\theta)$ 代入目标函数的梯度公式化简为期望形式，就能用蒙特卡洛采样来估算梯度。

更深入的理解：对数导数技巧的通用性

这种技巧不只用于策略梯度，而是一般参数化概率分布下求期望梯度的核心技巧：

$$\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}(x)} [f(x)] = \mathbb{E}_{x \sim p_{\theta}(x)} [f(x) \nabla_{\theta} \log p_{\theta}(x)] \quad (9)$$

在 Policy Gradient 里， $x = \tau$ ， $f(x) = R(\tau)$ ，因此自然用到了这一技巧。

3.3 第三步：展开 $\log p(\tau|\theta)$

回忆轨迹概率式 (3)。取对数：

$$\log p(\tau|\theta) = \log p(s_0) + \sum_{t=0}^T \log \pi_{\theta}(a_t|s_t) + \sum_{t=0}^T \log P(s_{t+1}|s_t, a_t) \quad (10)$$

对 θ 求梯度：

$$\nabla_{\theta} \log p(\tau|\theta) = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \quad (11)$$

关键点：

- $\log p(s_0)$ ：初始状态分布，与 θ 无关，梯度为 0
- $\sum \log P(s_{t+1}|s_t, a_t)$ ：状态转移概率，由环境决定，与 θ 无关，梯度为 0
- 只有 $\sum \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$ 保留下来，这是策略网络对参数的梯度

3.4 第四步：得到原始梯度公式

代回期望公式 (8)：

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\left(\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \right) \left(\sum_{k=0}^T r_k \right) \right] \quad (12)$$

直观理解：整条轨迹的总奖励 $R(\tau)$ 作为权重，同时强化轨迹中出现的所有动作。

思考 3.2—怎么能不分具体动作的好坏，依照轨迹级的奖励信号，让所有的动作输出的概率统一加减？

答：信用分配问题

这在强化学习中叫做**信用分配问题**（Credit Assignment Problem）。但即使这样“一刀切”地更新，在统计学上模型优化依旧能够奏效——好动作 A 所在的轨迹总奖励高，被强化很多遍；坏动作 B 所在的轨迹总奖励低，被强化很少。最终 A 的概率被拉高，B 的概率被相对拉低。

但这样的算法样本效率低，于是引发了后续更精细的更新算法：其一是引入下文的“因果律”，其二是 Actor-Critic、PPO 等采用了优势函数来评估每一步动作的好坏，从而精确调控某一步动作的概率。

4 因果律与梯度简化

4.1 问题：过去奖励不应影响当前动作

上述公式有一个明显的问题：用整条轨迹的总奖励去奖励或惩罚**每一个**动作。但第 t 步做出的决策，不应该被第 t 步之前就已经发生的奖励所影响。

可以严格证明（见下节），对于 $k < t$ ：

$$\mathbb{E}_{\tau} [\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot r_k] = 0 \quad (13)$$

这意味着过去奖励对当前动作的梯度贡献在期望意义下为零。

4.2 严格证明

我们需要证明：对于任意 $k \leq t$ （即奖励 r_k 在动作 a_t 发生之前），有式 (13) 成立。

第一步：轨迹概率的拆解。

将轨迹 τ 拆分为三部分：

- $\tau_{:t} = (s_0, a_0, \dots, s_t)$ ： t 时刻之前的历史

- a_t : t 时刻采取的动作
- $\tau_{t+1:} = (r_{t+1}, s_{t+1}, \dots, r_T, s_T)$: t 时刻之后的轨迹

整条轨迹的联合概率:

$$p(\tau|\theta) = p(\tau_{:t}|\theta) \cdot \pi_\theta(a_t|s_t) \cdot p(\tau_{t+1:}|\tau_{:t}, a_t, \theta) \quad (14)$$

第二步：利用重期望公式。

$$\mathbb{E}_\tau[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot r_k] = \mathbb{E}_{\tau_{:t}, a_t}[\mathbb{E}_{\tau_{t+1:}|\tau_{:t}, a_t}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot r_k]] \quad (15)$$

第三步：识别常数。

对于给定的历史 $\tau_{:t}$ 和动作 a_t ，奖励 r_k （其中 $k \leq t$ ）和 $\nabla_\theta \log \pi_\theta(a_t|s_t)$ 都是**已知常数**，与后续轨迹 $\tau_{t+1:}$ 无关。

内部条件期望可以将它们作为常数提出：

$$\mathbb{E}_{\tau_{t+1:}|\tau_{:t}, a_t}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot r_k] = \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot r_k \quad (16)$$

代回外层期望：

$$= \mathbb{E}_{\tau_{:t}, a_t}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot r_k] \quad (17)$$

第四步：展开积分。

$$= \int_{\tau_{:t}} \int_{a_t} p(\tau_{:t}|\theta) \cdot \pi_\theta(a_t|s_t) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot r_k \, da_t \, d\tau_{:t} \quad (18)$$

第五步：对数导数还原。

利用恒等式 $\pi_\theta \cdot \nabla_\theta \log \pi_\theta = \nabla_\theta \pi_\theta$ ：

$$= \int_{\tau_{:t}} \int_{a_t} p(\tau_{:t}|\theta) \cdot \nabla_\theta \pi_\theta(a_t|s_t) \cdot r_k \, da_t \, d\tau_{:t} \quad (19)$$

将 r_k （与 a_t 无关）移到外层积分：

$$= \int_{\tau_{:t}} p(\tau_{:t}|\theta) \cdot r_k \cdot \left(\int_{a_t} \nabla_\theta \pi_\theta(a_t|s_t) \, da_t \right) d\tau_{:t} \quad (20)$$

第六步：核心戏法——全空间概率积分为 1。

根据微积分基本定理，将梯度符号提到积分号外：

$$\int_{a_t} \nabla_\theta \pi_\theta(a_t|s_t) \, da_t = \nabla_\theta \left(\int_{a_t} \pi_\theta(a_t|s_t) \, da_t \right) \quad (21)$$

因为策略 $\pi_\theta(a_t|s_t)$ 是一个合法的概率分布，它对所有可能动作的积分（离散为求和）恒等于 1：

$$\int_{a_t} \pi_\theta(a_t|s_t) \, da_t = 1 \quad (22)$$

所以：

$$\int_{a_t} \nabla_{\theta} \pi_{\theta}(a_t | s_t) da_t = \nabla_{\theta}(1) = 0 \quad (23)$$

第七步：最终结果。
代回整个表达式：

$$= \int_{\tau:t} p(\tau:t | \theta) \cdot r_k \cdot 0 d\tau:t = 0 \quad (24)$$

证明完毕。

核心直观总结

因为全空间概率积分 $\int \pi_{\theta} = 1 \Rightarrow \nabla 1 = 0$ ，当 $k \leq t$ 时，奖励 r_k 不包含当前动作 a_t 之后的随机性，可以被当作常数提出。剩下的部分对整个动作空间积分，由于概率归一化，梯度贡献为零。

4.3 简化后的梯度公式

由于过去奖励对当前动作的梯度贡献为零，我们可以把奖励求和从 $k = 0$ 截断到 $k = t$ ，即从当前时间步开始计算未来累计奖励：

$$G_t \equiv \sum_{k=t}^T r_k \quad (25)$$

最终梯度公式（策略梯度定理）：

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right] \quad (26)$$

5 REINFORCE 算法

5.1 从公式到算法

根据上述推导，我们得到了梯度公式 (26)。

在代码实现中，我们无法穷举所有轨迹来计算完美的期望。让策略 π_{θ} 与环境交互，采样出一条或几条真实轨迹，用采样均值近似期望：

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \quad (27)$$

对于单个时间步 t ，它对整体梯度的贡献是：

$$\Delta \theta_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \quad (28)$$

这就是 REINFORCE 的核心更新公式。
符号说明：

- $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$: 策略网络输出的对数概率的梯度, 表示“在状态 s_t 下增加选择动作 a_t 的方向”
- G_t : 加权系数, 表示这个动作对整体奖励的贡献程度

5.2 完整算法流程（伪代码）

REINFORCE 算法伪代码

输入: 策略网络参数 θ , 超参数: 学习率 α , 折扣因子 γ

```

1. 初始化  $\theta$ 
2. repeat 直到收敛:
3.     使用当前策略  $\pi_{\theta}$  与环境交互, 采样一条完整轨迹
4.      $\tau = (s_0, a_0, r_1, s_1, \dots, s_T, a_T, r_T)$ 
5.     for  $t = 0$  to  $T$  do
6.         计算未来累计奖励:  $G_t = \sum_{k=t}^T \gamma^k r_{k+1}$ 
           (或折扣版本  $G_t = r_{t+1} + \gamma G_{t+1}$ )
7.         计算梯度增量:  $\Delta\theta = \Delta\theta + \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t$ 
8.     end for
9.     更新参数:  $\theta = \theta + \alpha \cdot \Delta\theta$ 
10. end repeat

```

在深度学习框架中, 通常将损失函数定义为 $\mathcal{L} = -\sum_t \log \pi_{\theta}(a_t|s_t) \cdot G_t$, 然后使用梯度下降 (即反向传播) 来更新参数。

5.3 直观理解

- $\log \pi_{\theta}(a_t|s_t)$: 表示“增加在状态 s_t 下选择动作 a_t 的概率”的方向
- G_t : 是权重, 决定这个方向的强度

如果 $G_t > 0$ (奖励为正), 梯度方向就是**增加**该动作的概率。如果 $G_t < 0$ (奖励为负), 梯度方向就是**减少**该动作的概率。

重要结论 5.1— *Policy Gradient* 的本质就是“好的动作多做, 坏的动作少做”。

5.4 REINFORCE 的方差问题

虽然 REINFORCE 在数学上优雅, 但存在严重的方差问题:

- G_t 是通过单条轨迹估计的随机量, 方差很大
- 每次采样不同的轨迹, 梯度估计波动剧烈
- 导致学习不稳定, 收敛速度慢

解决方案: 引入 Baseline (见下一章)。

6 方差减小技术——Baseline

6.1 全正/全负奖励也能工作的原因

问题：如果所有轨迹的奖励都是正的（如 $R(\tau_1) = 100, R(\tau_2) = 10$ ），梯度方向岂不是要增加所有动作的概率？

答案：这确实会发生，但因为概率总和恒等于 1（ $\sum \pi = 1$ ），当某个动作的概率被大幅增加时，其他动作的概率必然相对减少。

但这会导致方差极大：在随机采样下，采到哪个轨迹完全看运气，导致梯度估计不稳定。

6.2 Baseline 的正确性：不影响期望值

核心思想：从奖励中减去一个基线 $b(s)$ ，使得权重有正有负：

$$\nabla_{\theta} \mathcal{J} \approx \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (G_t - b(s_t)) \quad (29)$$

需要证明：减去 $b(s_t)$ 不会改变梯度的期望值，即 $\mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot b(s_t)] = 0$ 。

证明与第 4.2 节类似： $b(s_t)$ （与 a_t 无关）可以提到积分外，剩下的部分对整个动作空间积分，由于概率归一化 $\int \nabla_{\theta} \pi_{\theta} = \nabla_{\theta}(1) = 0$ ，整个积分为零。

重要结论 6.1—任何只依赖于状态 s 而不依赖于动作 a 的函数 $b(s)$ 作为基线，都不会改变梯度期望值，但能有效减小方差。

6.3 改进后的梯度估计

引入 Baseline 后，梯度公式变为式 (29)。

当 $G_t > b(s_t)$ 时（表现高于平均水平），权重为正，动作被强化；当 $G_t < b(s_t)$ 时（表现低于平均水平），权重为负，动作被抑制。

常用基线：当前状态的值函数 $V^{\pi}(s_t)$ 。

引入折扣因子 γ 后的完整公式：

$$\nabla_{\theta} \mathcal{J} \approx \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \left(\sum_{k=t}^T \gamma^{k-t} r_k - b(s_t) \right) \quad (30)$$

6.4 为什么 baseline 常用 $V^{\pi}(s)$ 而不是 $Q^{\pi}(s, a)$ ？

理论上：用 $Q(s, a)$ 作为基线也是可以的，这实际上是优势函数 $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ 的形式。

工程实践中：用 $V(s)$ 更常见，原因如下：

1. $V(s)$ 只依赖于状态，任何只依赖状态的函数都可以作为基线而不改变期望值；而 $Q(s, a)$ 依赖动作，引入它会改变期望。
2. $V(s)$ 的估计更简单：只需要一个输出（标量），而 $Q^{\pi}(s, a)$ 需要对每个动作都有一个估计，对于大动作空间代价高昂。
3. 实际中通常估计 $V(s)$ 然后通过 TD-error 近似 $A(s, a)$ 。

总结

$V^\pi(s)$ 是工程上的最优选择——它满足基线的条件（只依赖状态），同时估计简单，且能有效减少方差。

7 连续动作空间的处理

7.1 离散 vs 连续的本质区别

Table 2: 离散与连续动作空间的对比

维度	离散动作空间	连续动作空间
神经网络输出	每个动作的概率值 ($p_1, p_2, \dots$)	分布参数 (μ, σ)
动作获取方式	<code>Categorical(p).sample()</code>	<code>Normal(μ, σ).sample()</code>
算法核心	增大高分动作的概率值	改变分布形状，使高分动作区域的密度变大
计算复杂性	随动作数量线性增加	与动作数量无关

7.2 神经网络输出分布参数

在连续动作空间中，我们不再让神经网络输出每个动作的概率（因为无法枚举无限个动作），而是输出**概率分布的参数**。

高斯策略（最常用）：

$$\pi_\theta(a|s) = \mathcal{N}(a | \mu(s), \sigma^2(s)) \quad (31)$$

其中 $\mu(s)$ 和 $\sigma(s)$ 由神经网络输出：

- 输入：状态 s
- 输出：均值 $\mu(s)$ 和标准差 $\sigma(s)$

为什么用高斯分布？

- 高斯分布是自然且通用的连续分布
- 数学性质良好，对数概率的梯度有解析形式
- 可以通过调整 μ 和 σ 来控制探索程度

7.3 高斯策略的对数概率梯度

对于高斯策略 $\pi_\theta(a|s) = \mathcal{N}(\mu, \sigma^2)$ ，对数概率为：

$$\log \pi_\theta(a|s) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(a - \mu)^2}{2\sigma^2} \quad (32)$$

对 θ 求梯度：

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \frac{(a - \mu)}{\sigma^2} \nabla_{\theta} \mu - \left(\frac{(a - \mu)^2}{\sigma^2} - 1 \right) \nabla_{\theta} \sigma \quad (33)$$

注意：这里的 $\nabla_{\theta} \mu$ 和 $\nabla_{\theta} \sigma$ 是神经网络的梯度，由反向传播计算。

7.4 为什么不需要遍历无穷动作空间？

问题：在连续动作空间中，理论上存在无穷多个动作，如果要对所有动作计算概率积分（或求和），岂不是无法操作？

答案：策略梯度的巧妙之处——**我们不计算积分，只进行采样。**

策略梯度公式中的期望：

$$\nabla_{\theta} \mathcal{J} = \mathbb{E}_{s \sim d^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot G_t] \quad (34)$$

在代码实现中：

1. **采样：**根据当前策略 π_{θ} 采样一个动作 a_t
2. **计算：**计算该动作的 $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$
3. **加权：**乘以 G_t 作为梯度估计

我们不需要遍历整个动作空间，只需要计算“实际执行的那个动作”的梯度。

把连续策略想象成“捏橡皮泥”：

- 初始状态：分布形状扁平（高熵），智能体在乱动（随机探索）
- 采样与奖励：智能体根据当前 μ 和 σ 采样出一个动作 a_{sample}
- 计算梯度：如果 G_t 很大，梯度会把均值 μ **拉向** a_{sample} ，并可能**减小** σ （让分布变窄）
- 如果 G_t 很小，梯度会把均值 μ **推离** a_{sample}

核心结论

我们不是计算整个概率空间的分布，而是通过不断地在特定点采样，通过梯度下降告诉神经网络：“在这个状态 s 下，刚才采样的那个动作 a 效果不错（或很差），请把那个点附近的概率密度调高（或调低）。”

8 数值小例子

让我们用一个完整的例子来展示梯度计算和参数更新的过程。

8.1 环境设定

- **状态空间：** $S = \{s_1, s_2\}$ （两个状态）
- **动作空间：** $A = \{a_1, a_2\}$ （两个动作）
- **折扣因子：** $\gamma = 1$ （非折扣）
- **目标：**最大化期望累计奖励

8.2 初始策略参数

使用 softmax 策略：

$$\pi_{\theta}(a|s) = \frac{e^{\phi(s,a)^T \theta}}{\sum_{a'} e^{\phi(s,a')^T \theta}} \quad (35)$$

设特征向量 $\phi(s, a)$ ： $\phi(s_1, a_1) = 0$, $\phi(s_1, a_2) = 1$, $\phi(s_2, a_1) = 0$, $\phi(s_2, a_2) = 1$ 。

初始参数： $\theta = [0, 0]$ (两个状态的权重初始化为 0)。

初始策略 ($w = 0$ 时两个动作概率均为 0.5)：

- 在 s_1 下： $\pi(a_1|s_1) = 0.5$, $\pi(a_2|s_1) = 0.5$
- 在 s_2 下： $\pi(a_1|s_2) = 0.5$, $\pi(a_2|s_2) = 0.5$

8.3 单条轨迹采样

假设我们采样到以下轨迹：

$$\tau = (s_1, a_2, r_1 = 1, s_2, a_1, r_2 = 0) \quad (36)$$

- $t = 0$ ：在 s_1 选择 a_2 ，获得 $r_1 = 1$ ，转移到 s_2
- $t = 1$ ：在 s_2 选择 a_1 ，获得 $r_2 = 0$ ，轨迹结束

计算 G_t ：

- $G_0 = r_1 + r_2 = 1 + 0 = 1$
- $G_1 = r_2 = 0$

8.4 梯度计算逐项展开

策略梯度的计算公式：

$$\nabla_{\theta} \mathcal{J} \approx \sum_{t=0}^1 \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t \quad (37)$$

对于 softmax 策略：

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \phi(s, a) - \mathbb{E}_{a' \sim \pi_{\theta}}[\phi(s, a')] \quad (38)$$

步骤 1： 计算 $\nabla_{\theta} \log \pi_{\theta}(a_2|s_1)$

- $\phi(s_1, a_2) = [0, 1]$
- 期望特征： $0.5 \cdot [0, 0] + 0.5 \cdot [0, 1] = [0, 0.5]$

$$\nabla_{\theta} \log \pi_{\theta}(a_2|s_1) = [0, 1] - [0, 0.5] = [0, 0.5]$$

步骤 2： 计算 $\nabla_{\theta} \log \pi_{\theta}(a_1|s_2)$

- $\phi(s_2, a_1) = [0, 0]$
- 期望特征: $0.5 \cdot [0, 0] + 0.5 \cdot [0, 1] = [0, 0.5]$

$$\nabla_{\theta} \log \pi_{\theta}(a_1 | s_2) = [0, 0] - [0, 0.5] = [0, -0.5]$$

步骤 3: 组装梯度

$$\Delta\theta = [0, 0.5] \cdot 1 + [0, -0.5] \cdot 0 = [0, 0.5]$$

8.5 参数更新

使用学习率 $\alpha = 0.1$:

$$\theta_{\text{new}} = [0, 0] + 0.1 \cdot [0, 0.5] = [0, 0.05]$$

更新后的策略:

- 在 s_1 下: $\pi(a_1 | s_1) \approx 0.487$, $\pi(a_2 | s_1) \approx 0.513$
- 在 s_2 下: $\pi(a_1 | s_2) \approx 0.487$, $\pi(a_2 | s_2) \approx 0.513$

8.6 更新前后策略对比

Table 3: 更新前后策略概率对比

状态	动作	更新前 π	更新后 π	变化
s_1	a_1	0.500	0.487	-0.013
s_1	a_2	0.500	0.513	+0.013
s_2	a_1	0.500	0.487	-0.013
s_2	a_2	0.500	0.513	+0.013

解释: 由于 $G_0 = 1 > 0$, 动作 a_2 在 s_1 下被强化 (概率增加); $G_1 = 0$, 动作 a_1 在 s_2 下既未被强化也未被抑制, 但因为概率归一化, 它相对被降低。

注 8.1— 实际中通常使用 *baseline* (如 $V^{\pi}(s)$) 来使权重有正有负, 使得学习更稳定。这个例子中没有加入 *baseline* 是为了演示简单。

9 代码实现与实战项目

9.1 PyTorch REINFORCE 完整代码

以下是基于 PyTorch 实现的经典 REINFORCE 算法代码, 以 Gym 库中的 CartPole-v1 为例进行训练:

```

1 import gymnasium as gym
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.nn.functional as F
6 import torch.optim as optim
7 from torch.distributions import Categorical
8
9 # 1. 定义策略网络 (Policy Network)
10 class PolicyNet(nn.Module):
11     def __init__(self, state_dim, action_dim, hidden_dim=128):
12         super(PolicyNet, self).__init__()
13         self.fc1 = nn.Linear(state_dim, hidden_dim)
14         self.fc2 = nn.Linear(hidden_dim, action_dim)
15
16     def forward(self, x):
17         x = F.relu(self.fc1(x))
18         return F.softmax(self.fc2(x), dim=-1)
19
20 # 2. 定义 REINFORCE 算法智能体
21 class REINFORCE:
22     def __init__(self, state_dim, action_dim, lr=0.001, gamma
23         =0.99):
24         self.policy_net = PolicyNet(state_dim, action_dim)
25         self.optimizer = optim.Adam(self.policy_net.parameters(),
26             lr=lr)
27         self.gamma = gamma
28         self.log_probs = []
29         self.rewards = []
30
31     def choose_action(self, state):
32         state = torch.from_numpy(state).float().unsqueeze(0)
33         probs = self.policy_net(state)
34         m = Categorical(probs)
35         action = m.sample()
36         self.log_probs.append(m.log_prob(action))
37         return action.item()
38
39     def update(self):
40         discounted_rewards = []
41         G = 0
42         for r in reversed(self.rewards):
43             G = r + self.gamma * G
44             discounted_rewards.insert(0, G)
45         discounted_rewards = torch.tensor(discounted_rewards,
46             dtype=torch.float32)
47         # 标准化 (批次级基线减法与方差缩放)
48         if len(discounted_rewards) > 1:
49             discounted_rewards = (discounted_rewards -
50                 discounted_rewards.mean()) /

```

```

47             (discounted_rewards.std() + 1e
48               -9)
49     policy_loss = []
50     for log_prob, G_t in zip(self.log_probs,
51                             discounted_rewards):
52         policy_loss.append(-log_prob * G_t)
53     self.optimizer.zero_grad()
54     policy_loss = torch.cat(policy_loss).sum()
55     policy_loss.backward()
56     self.optimizer.step()
57     self.log_probs = []
58     self.rewards = []
59
60 def train():
61     env = gym.make('CartPole-v1')
62     state_dim = env.observation_space.shape[0]
63     action_dim = env.action_space.n
64     agent = REINFORCE(state_dim, action_dim, lr=0.001)
65     for episode in range(500):
66         state, _ = env.reset()
67         episode_reward = 0
68         while True:
69             action = agent.choose_action(state)
70             next_state, reward, terminated, truncated, _ = env.
71                 step(action)
72             agent.rewards.append(reward)
73             state = next_state
74             episode_reward += reward
75             if terminated or truncated:
76                 break
77         agent.update()
78         if (episode + 1) % 20 == 0:
79             print(f"Episode {episode+1:3d} | Reward: {
80                 episode_reward:.0f}")
81     env.close()
82
83 if __name__ == "__main__":
84     train()

```

Listing 1: REINFORCE 算法 PyTorch 实现

Table 4: 代码核心要点解读

代码段	说明
<code>Categorical(probs).sample()</code>	无偏采样，维持探索能力
<code>for r in reversed(self.rewards) + G = r + self.gamma * G</code>	逆向计算 G_t ，完美契合因果律
<code>(discounted_rewards - mean) / (std + 1e-9)</code>	批次级基线标准化，大幅降低方差
<code>-log_prob * G_t</code>	损失函数，梯度上升方向（即最小化负方向）

9.2 推荐实战项目作业

9.2.1 作业 1：车杆平衡升级版——引入连续动作控制（入门）

项目环境： Gym 中的 Pendulum-v1 或 MountainCarContinuous-v0。

作业目标： 将上面的离散 REINFORCE 代码改造成连续动作空间版本。

核心挑战：

1. 修改 PolicyNet 的输出，使其不再输出 Softmax 概率，而是输出高斯分布的均值 μ 和标准差 σ
2. 利用 PyTorch 的 `torch.distributions.Normal` 代替 `Categorical` 进行动作采样和对数概率计算

意义： 亲身体会神经网络如何通过“改变高斯分布的形状”来控制连续动作。

9.2.2 作业 2：经典街机小游戏——像素输入下的 Pong（进阶）

项目环境： Gym 中的 Pong-v4。

作业目标： 智能体通过观察游戏界面的像素画面（图像输入），学习如何控制挡板击球。

核心挑战：

1. **高维输入：** 策略网络的前端需要从全连接层（FC）换成卷积神经网络（CNN）来提取图像特征
2. **稀疏奖励：** 球漏掉了扣 1 分，接住并赢了得 1 分，绝大多数时候奖励是 0

意义： 这是 RL 历史上最经典的学术里程碑项目之一，能极大地锻炼调参和处理高维特征的能力。

9.2.3 作业 3：手写高维连续动作挑战——双足机器人行走（高阶）

项目环境： Gym 中的 BipedalWalker-v3。

作业目标： 控制一个拥有 4 个连续动作关节（两条腿）的机器人向前行走，跨越障碍，且不能摔倒。

核心挑战：

1. **高维动作方差爆发**：基本 REINFORCE 算法会因为方差太大而极难收敛
 2. **算法升级要求**：需要引入 Actor-Critic 架构，或实现 PPO 限制策略更新步长
- 意义**：真正迈入现代工业级强化学习（机器人控制、自动驾驶）的门槛。

实验建议

调参的两个命门：学习率和奖励的标准化（Normalization）是决定 Policy Gradient 能否成功收敛的两个最关键的命门！

10 工程实践：标准化与调参

10.1 批次级基线减法与方差缩放

在 REINFORCE 代码中，有这样一行精妙的标准化操作：

```
1 discounted_rewards = (discounted_rewards - discounted_rewards.  
    mean())  
2                      / (discounted_rewards.std() + 1e-9)
```

这行代码背后的数学操作，被称为“**批次级基线减法与方差缩放**”（Batch-level Baseline Subtraction and Variance Scaling）。

10.1.1 减去均值：动态基线的物理引入

代码中的均值 `mean()` 在统计学上，就是当前策略在这些状态下的“平均预期收益”，充当了状态价值 $V(s)$ 的一个**经验估计值**。

- 如果 $G_t > \text{mean}()$ ：相减后结果为正，算法精准**强化**该动作
- 如果 $G_t < \text{mean}()$ ：相减后结果为负，算法精准**惩罚**该动作

通过减去均值，把所有的回报拉到了以 0 为中心的对称区间，实现了“好的方向权重为正去强化，坏的方向权重为负去避免”。

10.1.2 除以标准差：梯度平稳化

除以标准差 (`std()`) 在统计学上叫**数据标准化**（Standardization）。它把回报的波动幅度限制在了一个单位方差（Variance = 1）的稳定世界里：

$$G_t^{\text{scaled}} = \frac{G_t - \mu}{\sigma} \quad (39)$$

无论这局游戏最终的总分是 10 分还是 10000 分，经过标准差的缩放后，高出平均水平的动作其权重基本都在 +1 到 +2 之间，低于平均水平的也都在 -1 到 -2 之间。这带来了两个巨大好处：

1. **梯度极其平稳**：神经网络接收到的激活信号和梯度回传大小在整个训练生命周期里是恒定、可控的

2. 免去了为不同环境单独调参的痛苦：经过标准差一除，在网络眼里全都变成了统一尺度的“标准信号”

+ $1e-9$ 是为了在标准差为 0 时充当一个“安全垫”，确保分母永远不为 0，保证代码的鲁棒性。

10.1.3 举例：动手算一遍

设定场景：假设智能体在某个濒死状态 s ，有两个动作可选：

- **动作 A (好动作)：**能挽救局面，多撑 20 步
- **动作 B (差动作)：**彻底搞砸，只能多撑 2 步

收集到 4 次采样的 G_t 数据：[110, 90, 12, 8]。

方案一：不加 Baseline

- 权重：[110, 90, 12, 8]，均值 $\mu = 55$
- 方差 $\sigma^2 = 2077$

方案二：加入 Baseline (减去均值 55)

- 新权重：[55, 35, -43, -47]，新均值 $\mu_{\text{new}} = 0$
- 方差 $\sigma_{\text{new}}^2 = 2077$ (与方案一相同!)

注意

仅仅做“减去均值（平移）”的操作，在数学上是绝对无法改变一组数据内部的方差的！

但真正的“降低方差”体现在哪里？

关键在于：不是这批固定回报数值的方差，而是“由于采样不均匀，导致网络在不同批次间更新参数时的梯度方差”减小了！

假设下一轮训练中，智能体尝试了 3 次全是差动作 B (12, 8, 10)，只执行了 1 次好动作 A (110)：

- **不加 Baseline：**所有权重都是正数 [110, 12, 8, 10]。动作 A 和动作 B 都在被往上拉，梯度方向混乱
- **加入 Baseline：**均值 = 35，权重变为 [75, -23, -27, -25]。动作 A 的权重是 +75 (强化)，3 个动作 B 的权重全是负数 (打压)

这就是 Baseline 降低梯度的“方差”的真正本质——它通过引入相对好坏，锚定了梯度的更新方向，让算法能够极其平稳地收敛。

10.2 PG 算法是 On-policy 吗？

原生的策略梯度 (如 REINFORCE) 是严格的 On-policy (同策略) 算法，绝对不能直接用旧数据更新！

策略梯度的目标函数 $\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$ 里，那个期望符号 $\mathbb{E}$ 要求：轨迹 τ 必须是由当前最新参数 θ 的策略网络产生的。

要想让 PG 算法变成 Off-policy 并复用旧数据，必须在数学上引入重要性采样 (Importance Sampling)。这正是 PPO (Proximal Policy Optimization) 算法的核心机理。

10.3 学习率 (Learning Rate) 的设置讲究

Policy Gradient 对学习率 lr 的敏感度远超 DQN 等 Value-based 算法。

为什么不能设太大？(策略瓦解)

在 PG 中，如果 lr 过大，网络参数发生剧烈跳跃，可能会让策略直接走向极端（例如某个状态下，动作 A 的概率瞬间被拉到 99.9%）。因为它是 On-policy 的，下一轮它就会以 99.9% 的概率疯狂采集动作 A 的数据。这意味着智能体彻底丧失了探索能力，直接陷入死胡同。这种现象叫做**策略瓦解** (Policy Collapse)。

为什么不能设太小？

PG 算法本身的方差极大，样本利用率极低。如果 lr 设得太小（比如 10^{-5} ），模型在海量噪声梯度中会像蜗牛一样爬行。

最佳实践：

- **标准范围：** 通常在 10^{-4} 到 10^{-3} 之间
- **强烈推荐结合 Adam 优化器：** Adam 的自适应动态调整步长特性非常契合 PG
- **现代解法：** 如果觉得 lr 太难调，建议直接换成 PPO 算法。PPO 内部有裁剪机制，相当于给学习率加上了“安全带”

10.4 折扣因子 (Gamma) 的设置讲究

折扣因子 $\gamma \in [0, 1]$ 决定了智能体眼光的长远程度。它在数学上引发了强化学习中最经典的“**偏差-方差权衡**” (Bias-Variance Trade-off)。

当 γ 趋近于 1 (如 0.99) —— **眼光长远**：适用于需要长线布局的任务（如围棋、迷宫）。代价（高方差）：未来的轨迹拉得非常长，后面几十步、上百步环境带来的随机性噪声会全部通过逆向叠加，汇聚到开局的 G_0 中。这会导致 G_t 的方差惊人地爆发。

当 γ 较小 (如 0.9) —— **短视近利**：适用于即时反馈强、一击即中的任务（如打地鼠、吃金币）。代价（高偏差）：由于远期的奖励被指数级衰减抹杀了，智能体在数学上被逼成了一个“近视眼”。

最佳实践：

- **常规默认值：** 绝大多数强化学习任务默认 $\gamma = 0.99$
- **Gamma 课程学习：** 在极度困难、动作序列超长的任务中，训练刚开始时设 $\gamma = 0.9$ ，随着训练推进，逐渐把 γ 调高到 0.99

A 附录

A.1 核心公式汇总

轨迹概率：

$$p(\tau|\theta) = p(s_0) \prod_{t=0}^T \pi_{\theta}(a_t|s_t) P(s_{t+1}|s_t, a_t)$$

目标函数：

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

策略梯度定理：

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t \right]$$

REINFORCE 更新：

$$\Delta \theta = \alpha \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$$

带 Baseline 的更新：

$$\Delta \theta = \alpha \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (G_t - b(s_t))$$

A.2 符号表

Table 5: 符号表

符号	含义
$\pi_{\theta}(a s)$	参数化策略，在状态 s 下选择动作 a 的概率
τ	轨迹 (Trajectory)，完整的交互序列
G_t	从时间步 t 开始的未来累计折扣奖励
$V^{\pi}(s)$	状态值函数，在策略 π 下状态 s 的期望累计奖励
$Q^{\pi}(s, a)$	状态-动作值函数，在策略 π 下状态 s 采取动作 a 的期望累计奖励
$A^{\pi}(s, a)$	优势函数， $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$
δ_t	TD-error, $r_t + \gamma V(s_{t+1}) - V(s_t)$
F	Fisher 信息矩阵
$\nabla_{\text{nat}} \mathcal{J}$	自然梯度
γ	折扣因子
σ	标准差 (Standard Deviation)

作者信息

基本信息： 杨骞玺 - 06 年 - 北京理工大学 - 机器人工程专业

自我介绍： 一位具身领域的学徒，未来想长期从事具身方向的科研和创业，用机器人替代人类重复的体力劳动。欢迎大家加我 vx 交朋友，我很丰富，无法简介；)

vx： Y1832800